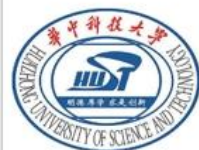


软件工程

第2章 需求获取

2.2 软件需求的表示方法



如何表示软件需求？

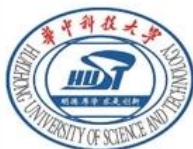
以自然语言描述 软件需求

问题何在？

◆ 不直观

◆ 需求项之间的关系深藏不露，难以发现

- ❑ 用户输入目的车站的名字，系统动态地显示匹配的目的地车站的清单。
- ❑ 在用户从清单中选定目的地车站之后，系统给出所有可能的乘车路线及每条路线的车票价格。
- ❑ 用户从中选定一条路线之后，系统显示车票价格并提示用户插入银行卡并输入身份证号。系统也可以接收来自身份证识别装置传入的身份证号。
- ❑ 系统检查银行卡及身份证号的有效性，检查银行卡余额是否足够支付票款。检查通过后，系统从银行卡扣款并在用户输入齐备后2秒之内弹出车票。



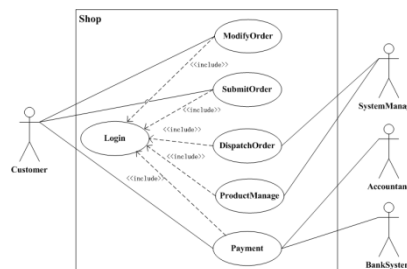
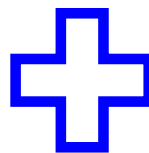
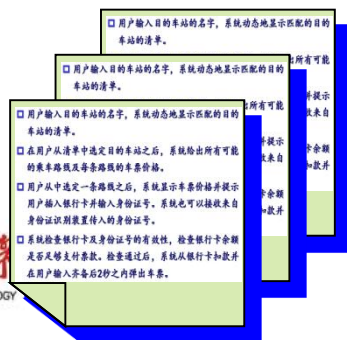
可否将文字与图形表示相结合 以描述软件需求？

□ 以文字分别描述各需求项

◆ 发挥文字在细节描述方面的优势

□ 以图形描述需求项之间的关系，直观地表现软件需求的整体结构

◆ 发挥图形表示直观、易理解的优势



软件需求的表示方法

1.

- 如何描述软件需求项?

2.

- 如何描述需求项之间的关系?

如何描述软件需求项？

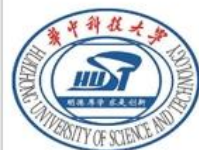
□ 如何描述功能需求项？

□ 如何描述非功能性需求项（质量需求项、约束性需求项）？

如何描述非功能性需求项？

□ 用简洁、清晰（无歧义）的自然语言句子或 短段落 描述非功能需求，例如：

- ◆ 系统在用户输入齐备后2秒之内弹出车票
- ◆ 目标软件必须在0.5秒内响应外部事件
- ◆ 针对任意目标地点，MCS必须在1秒内完成最优路径规划
- ◆



如何描述功能需求项？

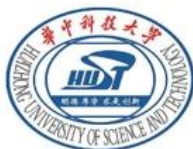
- 直观上，可将功能理解为输入-输出关系，于是，功能需求项可表示为：
 - ◆ 输入数据，以及对输入数据的约束或要求
 - ◆ 输出数据
 - ◆ 对输出数据的期望，以及对输入、输出数据之间关系的约束或要求

以输入-输出关系描述功能需求项的示例

□ 需求项名称：照相控制

- ◆ 输入数据：月球车的当前坐标、方位，照相目标区域的坐标
- ◆ 输出数据：月球车的方位调整指令、照相启动指令
- ◆ 要求：照相启动前，相机必须位于目标区域的正前方5米

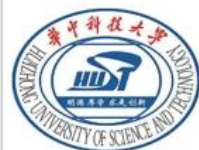
这样的描述方式存在什么问题？



找问题，探索更好的需求描述方式

□ 无法描述针对软件系统在各时间点上的行为期望：

◆ 在每步移动完成后，系统根据当前实际位置和规划路径发出下一步移动指令



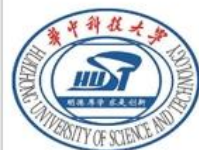
进一步的问题

□ 如何描述以下需求？

- ◆ 给定照相目标区域的坐标，控制月球车移动到区域正前方5米，启动照相
- ◆ 在移动过程中，**用户可随时暂停月球车的移动**，修正目标区域的坐标，然后要求MCS重新规划路径

输入-输出关系描述方式的缺陷

- 无法描述软件系统的动作序列
- 无法描述用户与软件系统之间的多次交互行为
 - ◆ **交互**是现代软件区别于传统的批处理软件的关键特征之一

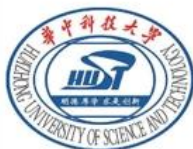


如何克服此缺陷？

□ 在需求项的描述文字中表现用户与软件系统（简称系统）之间的交互场景

- ◆ 用户提供输入数据1；
- ◆ 系统通过外部可见的动作1（例如，产生输出数据）来响应数据1；
- ◆ 用户提供输入数据2；
- ◆ 系统通过外部可见的动作2来响应数据2；
- ◆

用户、系统之间的交替动作序列



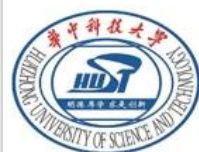
引入“用例”以描述功能需求项

□ 什么是用例（ use case）？

- ◆ 执行者(actor)与目标软件系统之间一次典型的交互动作序列，其效果就是执行者在系统的帮助下完成了某项业务功能，或达成了某项业务目标

□ 什么是执行者？

- ◆ **外部实体**（用户、设备、其他软件系统）在系统的交互过程中扮演的角色，它触发用例的执行，或者与软件系统交换信息并使用系统的功能



用例的示例

- 用例名：移动并照相
- 执行者：用户
- 交互动作序列：
 1. 用户指定照相目标区域的坐标；系统规划从当前位置到达照相目标区域正前方5米的路径；
 2. 系统发出移动指令；在移动完成后系统根据当前实际位置和规划路径发出下一步移动指令，如此重复直至到达路径终点；
 3. 到达终点后系统发出启动照相指令。



用例 与 输入-输出关系

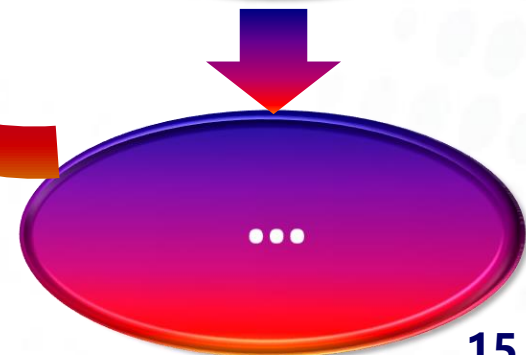
两种功能需求描述方式的比较



孤立、局部



交互、全局



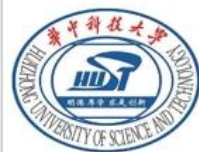
小结：软件需求的表示方法

1.

- 如何描述软件需求项 —— 用例
(交互动作序列)

2.

- 如何描述需求项(用例)之间的关系?



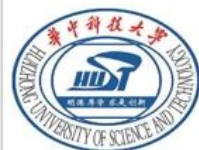
用例之间可能存在哪些关系？

□ 考察MCS中的三个用例：

- ◆ 移动至指定位置
- ◆ 移动并照相
- ◆ 移动并取样

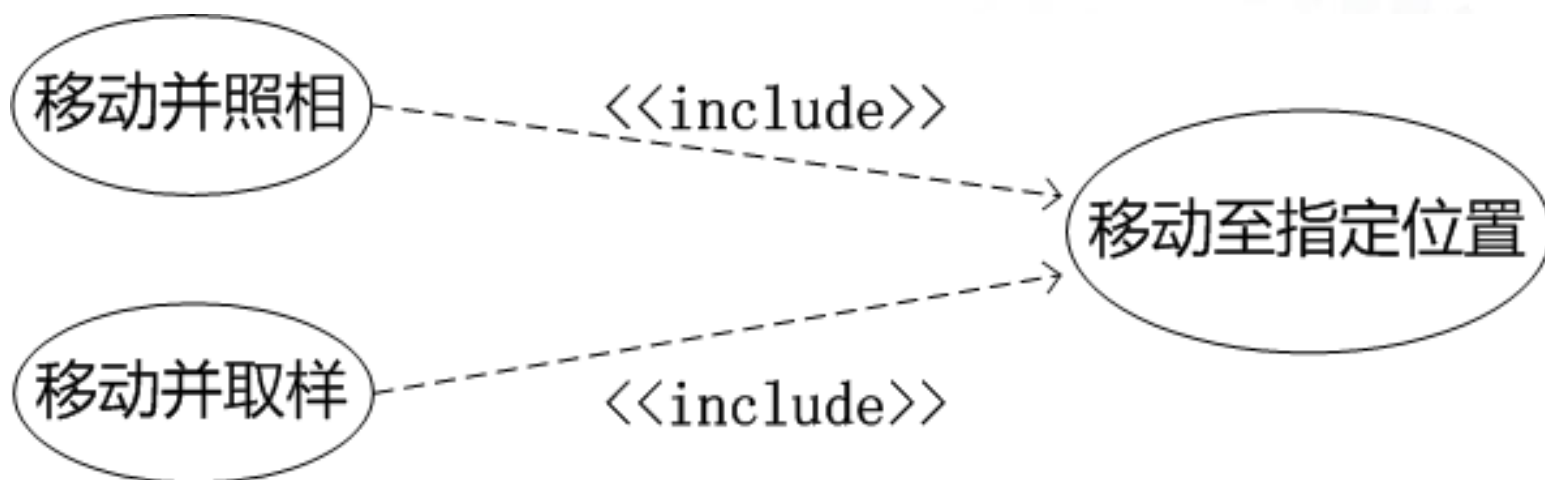
它们中以下动作序列相同：

1. 系统规划从当前位置到达指定位置的最优路径；
2. 系统发出移动指令；在移动完成后系统根据规划路径发出下一步移动指令，如此重复直至到达路径终点。



如何避免在用例中 重复描述相同的交互动作序列？

- 借鉴程序设计中**提取公共子函数**的方法，引入用例之间的“包含” (include) 关系

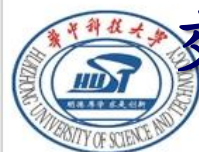


利用包含关系简化用例描述

- 用例名：移动并照相
- 执行者：用户
- 交互动作序列：
 1. 用户指定照相目标区域的坐标；
 2. 移动至指定位置（照相目标区域正前方5米）；
 3. 系统在到达终点后发出启动照相指令。
 4.

在用例名之下加下划线，表示引用该用例的

交互动作序列



除包含外，用例间是否还存在其他可能的关系？

- 再次考察“移动至指定位置”用例，如果在移动过程中导航相机在近前方发现障碍，必须紧急避开，如何描述重新规划路径的动作？
 - ◆ 在初次进行路径规划时，由于距离太远，无法精确地判定路径上远处的障碍
 - ◆ 在移动过程中针对新发现的近前方障碍即时调整路径，与路径规划中的障碍避绕功能构成双保险



一种可能的描述方案

□ 交互动作序列：

1. 系统规划从当前位置到达指定位置的最佳路径；
2. 系统发出移动指令；在移动完成后系统根据规划路径发出下一步移动指令，如此重复直至到达路径终点。
3. 如果导航相机在近前方1米内发现障碍，系统重新规划路径，再执行2。

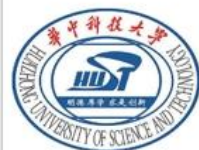
问题何在？是否可以改进？如何改进？

如何分离异常处理与正常处理逻辑？

□ 异常处理与正常处理逻辑混杂在一起，
导致 复杂、不清晰、难于理解

□ 怎么办？

◆ 借鉴程序设计中**结构化异常处理**机制，分离
异常处理与正常处理逻辑



程序设计中结构化异常处理示例

```
String parser = XMLResourceDescriptor.getXMLParserClassName();
SAXSVGDocumentFactory factory = new SAXSVGDocumentFactory(parser);
try {
    Document document;
    if (documentsMap.containsKey(uri)){
        document = documentsMap.get(uri);
    } else {
        document = factory.createDocument(uri);
        documentsMap.put(uri, document);
    }
    transcoder = new SimpleImageTranscoder(document);
    failedToLoadDocument = false;
} catch (IOException e) {
    Activator.logError("Error loading SVG file", e);
}
```

改进的描述方案1：分离异常处理与正常处理

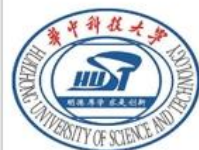
□ 基本交互动作序列：

1. 系统规划从当前位置到达指定位置的最佳路径；
2. 系统发出移动指令；在移动完成后系统根据规划路径发出下一步移动指令，如此重复直至到达路径终点。

□ 扩展交互动作序列：

2a. 如果导航相机在近前方1米内发现障碍

2a.1 系统重新规划路径，再执行2。



扩展交互动作的描述方法

□ 扩展交互动作序列：

2a. 如果导航相机在近前方1米内发现障碍

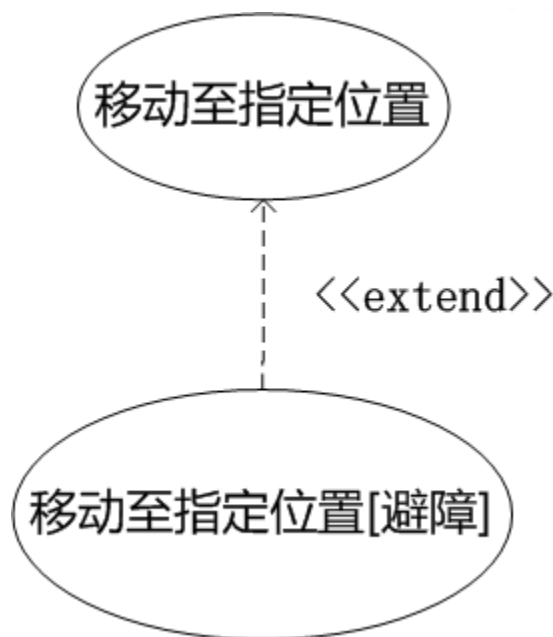
“2a”表示在基本动作2的过程中发生的首个（a）异常条件，“2b”表示第2个（b）异常条件，依此类推

2a.1 系统重新规划路径，再执行2。

“2a.1”表示异常处理的首个动作，“2a.2”表示第2个动作，依此类推

改进的描述方案2: 进一步分离异常处理与正常处理

- 将基本交互动作序列和扩展序列安排至两个用例，并在它们之间建立**扩展**（extend）关系

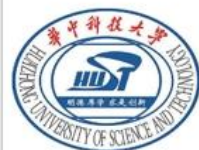


分别描述 被扩展用例用例 与 扩展用例

□ 用例名：移动至指定位置

□ 基本交互动作序列：

1. 系统规划从当前位置到达指定位置的最佳路径；
2. 系统发出移动指令；在移动完成后系统根据规划路径发出下一步移动指令，如此重复直至到达路径终点。



单独描述 扩展用例

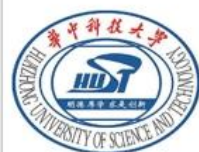
- 用例名：移动至指定位置[避障]
- 基本交互动作序列：同 移动至指定位置
- 扩展交互动作序列：
 - 2a. 如果导航相机在近前方1米内发现障碍
 - 2a.1 系统重新规划路径，再执行2。

两种描述方案如何取舍？

- 当正常处理和异常处理逻辑都比较简单、规模较小时，采用前一方案 —— 在用例中引入扩展交互动作序列；
- 否则，采用后一方案 —— 以扩展用例表示异常处理逻辑。

包含与扩展关系的比较

- 类似点：扩展用例 包含 被扩展用例的基本交互动作序列；
- 不同点：
 - ◆ 包含关系用于抽取公共的子动作序列
 - ◆ 扩展关系用于分离正常处理与异常处理的动作序列



小结：软件需求的表示方法

1.

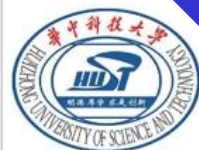
- 如何描述软件需求项 —— 用例（交互动作序列）

2.

- 如何描述需求项(用例)之间的关系 —— 包含 与 扩展

3.

- 是否需要描述执行者与用例之间的关系？

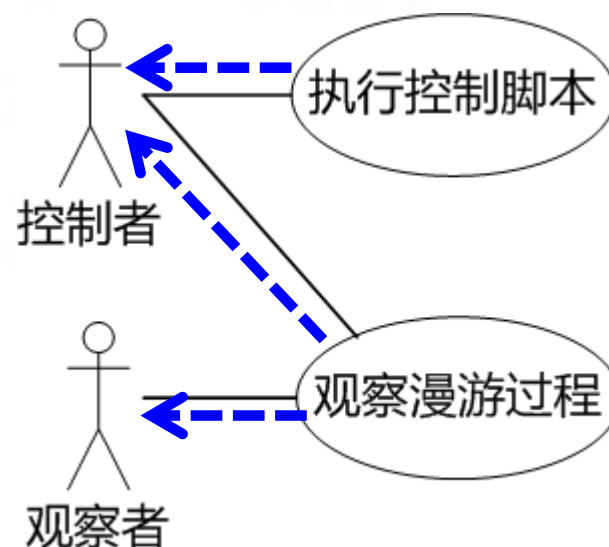


为什么描述执行者与用例之间的关系？

□ 不同的执行者使用不同的用例，显式表示特定执行者与相关用例之间的关系，

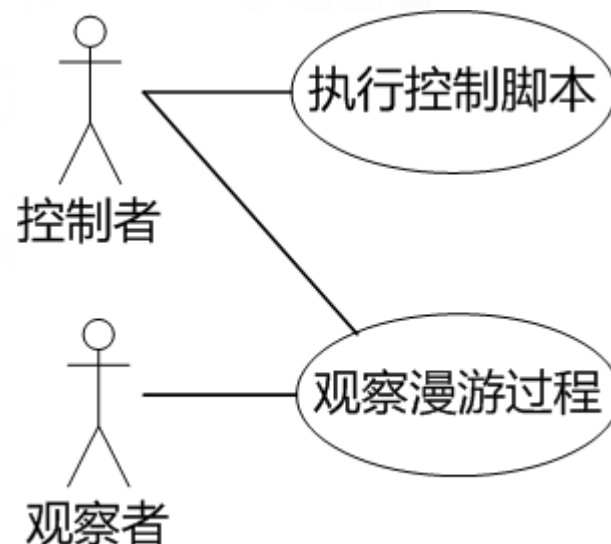
◆ 便于理解

◆ 支持需求项与需求源之间的追踪



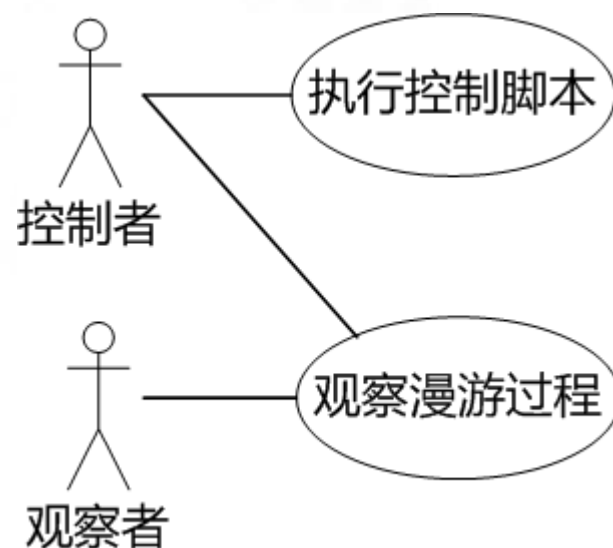
执行者与用例之间关系的语义

- 执行者触发用例的执行 —— **主动执行者**
- 执行者从用例获取信息：如果仅从用例获取信息，并不触发用例执行的执行者，称为**被动执行者**



执行者与用例之间关联边的方向

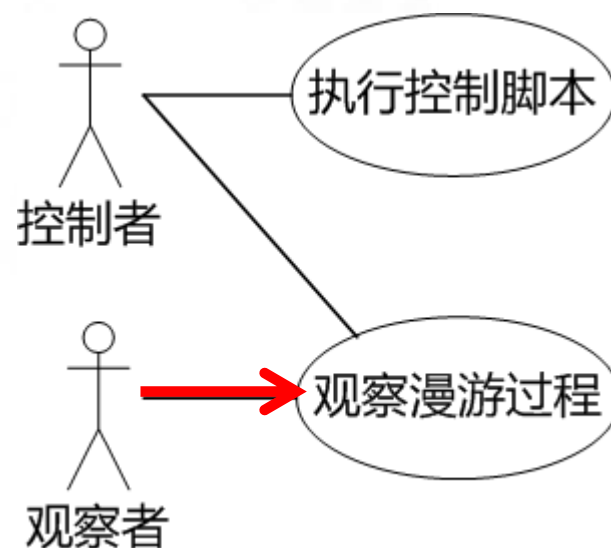
- 一般采用**无向边**表示执行者与用例之间的关联
 - ◆ 虽然触发执行是单向的，但执行者与用例之间的信息传递往往是双向的



但特殊情况下可采用有向边

□ 当有多个执行者与用例相连时，以有向边强调主要执行者

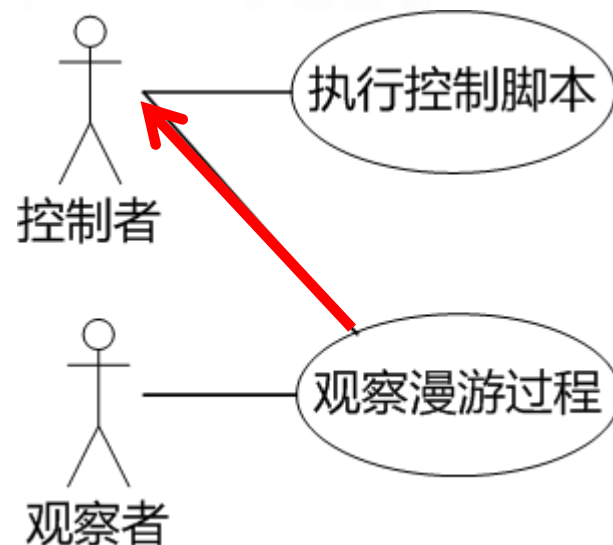
◆ 但主要执行者与用例之间的信息传递仍是双向的



可采用有向边的另一种特殊情况

□ 为强调被动执行者仅从用例获取信息，而不向用例提供信息

◆ 此时用例与执行者之间的信息传递是**单向的**（从用例传向执行者）



小结：软件需求的表示方法

1.

• 如何描述软件需求项 —— 用例（交互动作序列）

2.

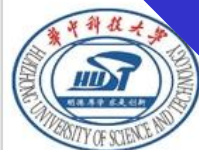
• 如何描述需求项之间的关系 —— 包含与扩展

3.

• 如何描述执行者与用例之间的关系 —— 触发执行，
信息传递

4.

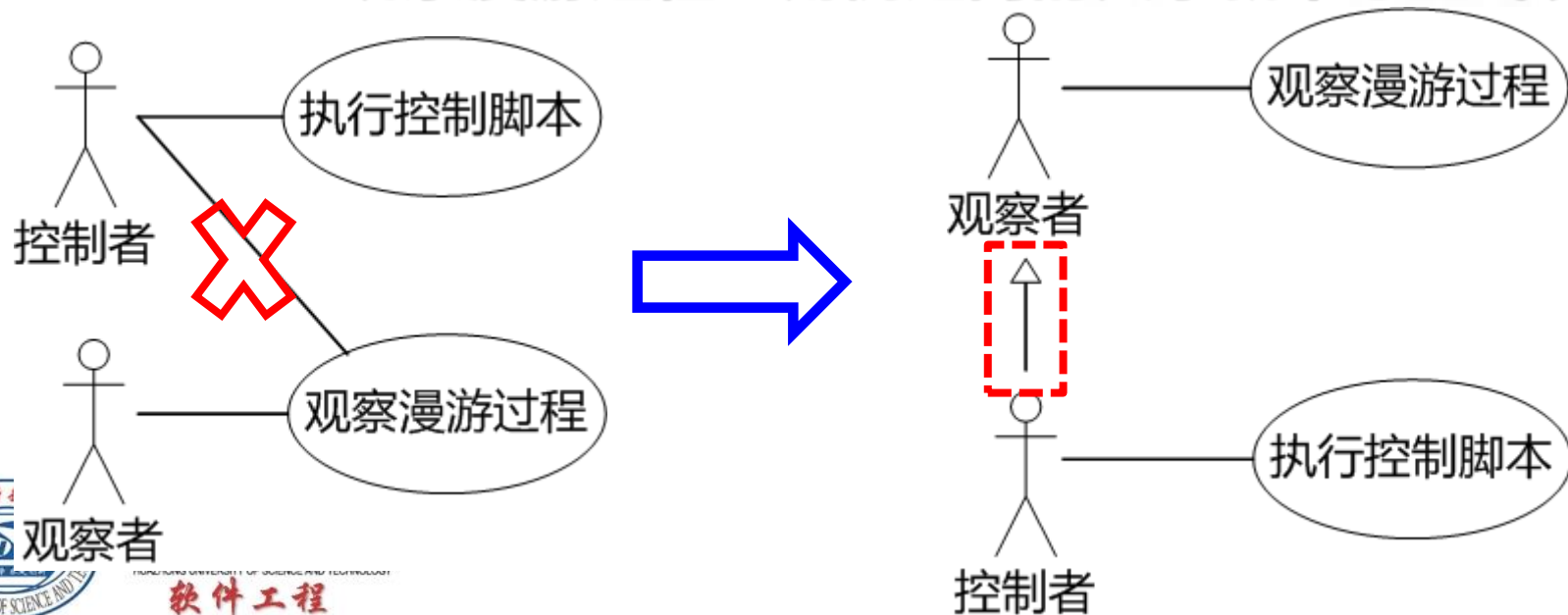
• 是否需要描述执行者之间的关系？



在执行者之间引入继承关系

□ 假如“控制者”是一种特殊的“观察者”，可在它们之间引入继承关系

◆ 因为“控制者”继承“观察者”，“控制者”与“观察漫游过程”用例之间的关系就不必显式表示



总结：软件需求的表示方法

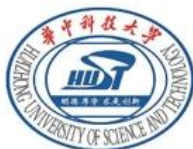
1.

- 如何描述软件需求项：用例（交互动作序列）

2.

- 用例间关系：包含与扩展
- 执行者间关系：继承
- 执行者与用例间关系：触发执行，信息传递

UML用例图



总结：软件需求的表示方法

